

HOLLOW KNIGHT: PABLO

RELAZIONE TECNICA

Bertoldini Luca - Giannattasio Lorenzo - Gianola Chiara - Frigerio Andrea

13/05/2026

---

1. Introduzione

1.1 Contesto del progetto

Il progetto Pablo nasce come videogioco platformer d'azione in due dimensioni, sviluppato interamente in linguaggio Java avvalendosi del framework libGDX. Il gioco è chiaramente ispirato a titoli del genere Metroidvania, in particolare a Hollow Knight, e ne riprende le meccaniche fondamentali: movimento fluido del personaggio, combattimento con nemici dotati di intelligenza artificiale propria, un boss finale, sistema di vita e di anima (soul), oltre a una mappa suddivisa in livelli collegati tra loro con transizioni automatiche.

L'applicativo è stato pensato come software stand-alone desktop, avviato tramite la classe Launcher.java, che configura la finestra di gioco in modalità fullscreen adattandosi alla risoluzione nativa del monitor host. Tutta la logica di gameplay, il rendering grafico, la gestione dell'audio e la fisica del personaggio vengono eseguiti sulla macchina locale senza necessità di infrastrutture server o connessione di rete.

1.2 Ispirazione e identità del progetto

Il nome del progetto ("Pablo") coincide con il nome del personaggio giocabile. La direzione artistica riprende l'estetica cupa e atmosferica dei platformer indie moderni: sfondi tematici, animazioni frame-by-frame per ogni nemico e per il protagonista, effetti visivi come lo screen shake durante gli attacchi del boss, e una colonna sonora distinta per ciascuna fase del gioco (menu, livello, boss). Il titolo completo del gioco è riportato nella configurazione del launcher come "HollowKnight-Pablo", confermando l'esplicita derivazione stilistica.

---

2. Obiettivi del progetto

L'obiettivo primario del progetto è la realizzazione di un videogioco desktop completo e funzionante, capace di dimostrare competenze concrete nei seguenti ambiti:

- Progettazione e implementazione di un game loop stabile e performante basato sul framework libGDX.
- Gestione di un sistema a schermate (screen management) con transizioni tra menu, intro, titolo, gameplay e pausa.
- Implementazione di fisica 2D personalizzata per il personaggio: gravità, salto, doppio salto, dash, collisioni con solidi tramite poligoni di collisione.
- Sviluppo di intelligenza artificiale per diversi tipi di nemici, ognuno con stati comportamentali distinti, e per un boss finale con meccaniche di attacco variabili.
- Creazione di un sistema di animazioni frame-by-frame altamente dettagliato e sincrono con il gameplay.
- Implementazione di un sistema di mappe collegate tramite transizioni laterali automatiche.
- Gestione di audio contestuale: musica di sottofondo, effetti sonori con cooldown per evitare sovrapposizioni indesiderate.
- Applicazione di pattern architetturali consolidati come il Singleton per la gestione globale di impostazioni, audio e stato del giocatore.

---

3. Tecnologie utilizzate

3.1 Java e libGDX

L'intero progetto è scritto in Java (compatibile con JDK 8+), linguaggio scelto per le sue garanzie di portabilità, tipizzazione statica e maturità dell'ecosistema. Il framework adottato è libGDX, una libreria open source per lo sviluppo di videogiochi multipiattaforma in Java che offre:

- Un game loop gestito automaticamente tramite le interfacce Screen e Game.

- Un sistema di rendering 2D basato su SpriteBatch e Stage di scene2d.
- Un motore di input unificato per tastiera, mouse e touch screen.
- Un sistema di audio che distingue tra tracce musicali (oggetti Music, adatti a brani lunghi in loop) e effetti sonori (oggetti Sound, adatti a clip brevi).
- Un sistema di caricamento asset tramite Gdx.files.internal(), che garantisce compatibilità cross-platform nella localizzazione dei file.
- Le classi TiledMap, TmxMapLoader e OrthogonalTiledMapRenderer per caricare e renderizzare mappe create con l'editor Tiled in formato .tmx.
- Viewport estendibili (ExtendViewport, ScreenViewport) per adattare il rendering a risoluzioni diverse senza distorsioni.

3.2 Backend desktop: LWJGL

Il progetto utilizza il backend LWJGL (Lightweight Java Game Library) tramite le classi LwjglApplication e LwjglApplicationConfiguration, che forniscono l'integrazione con OpenGL per il rendering accelerato via GPU, la gestione della finestra di sistema operativo e il ciclo di eventi di basso livello.

3.3 Tiled Map Editor

I livelli di gioco sono progettati con Tiled, editor open source per mappe isometriche e ortogonali. Le mappe vengono esportate in formato .tmx e caricate a runtime. Gli oggetti di collisione, i punti di spawn dei nemici e i trigger speciali (come il trigger del boss) sono definiti come layer di oggetti all'interno della mappa.

---

4. Struttura del progetto

4.1 Organizzazione dei package

Il codice sorgente è organizzato nella directory src/pablo/ e strutturato in package semantici:

- pablo/ — Contiene le classi di alto livello: entry point (Launcher, PabloGame), schermate (IntroScreen, TitleScreen, MenuScreen, LevelScreen, PauseScreen, OptionsScreen), singleton di supporto (GameSettings, SoundManager, PlayerState, MapGraph) e oggetti di scena (Object).
- pablo/framework/ — Contiene le classi base del framework custom costruito sopra libGDX: BaseActor, BaseGame, BaseScreen, TilemapActor. Queste classi astraggono le operazioni più frequenti e vengono estese da quasi tutti i componenti di gioco.

- pablo/entities/player/ — Contiene la classe Pablo (il personaggio giocabile) e l'enum Playerstate.
- pablo/entities/enemies/ — Contiene la classe astratta Enemy e le sottocartelle per ogni nemico: crawlid/, tiktik/, vengefly/, huskBully/, huskHornhead/, huskWarrior/, falseKnight/.
- pablo/entities/combat/ — Contiene la classe Hitbox, responsabile della rilevazione delle collisioni di attacco.

4.2 Flusso di avvio

L'avvio dell'applicazione parte da Launcher.main(), che rileva la risoluzione del monitor nativo tramite GraphicsEnvironment, configura la finestra in fullscreen e istanzia PabloGame. Quest'ultima estende BaseGame e nel metodo create() imposta come schermata attiva IntroScreen, dando il via alla sequenza di avvio:

- Launcher → PabloGame → IntroScreen → MenuScreen → LevelScreen

4.3 Pattern architetturali

Il progetto fa uso sistematico del pattern Singleton nelle classi di gestione globale:

- GameSettings.get() — Impostazioni audio e display, con persistenza tramite Preferences di libGDX.
- SoundManager.get() — Gestione centralizzata di musica ed effetti sonori.
- PlayerState — Salvataggio e ripristino dello stato del giocatore tra transizioni di mappa (implementato con campi statici).
- MapGraph — Registro statico delle connessioni tra mappe, inizializzato in un blocco static {}.

Il pattern State Machine è diffuso nella gestione dei comportamenti: sia il personaggio (Playerstate) sia ogni nemico (es. CrawlidState, FalseKnightState) sono governati da enum di stato e metodi tick dedicati.

---

5. Analisi tecnica del codice

5.1 MapGraph.java — Sistema di collegamento tra livelli

La classe MapGraph implementa un registro statico che mappa ogni livello del gioco alle aree adiacenti. È la componente architetturale centrale che rende possibile la navigazione orizzontale tra mappe multiple senza interruzioni di gameplay.

Struttura dati

Il registro è basato su una HashMap<String, MapNode> dove la chiave è il path del file .tmx della mappa e il valore è un oggetto MapNode:

- private static final Map<String, MapNode> NODES = new HashMap<>();

La classe interna MapNode è volutamente minimale: contiene il path della mappa e due riferimenti opzionali ai vicini:

```
public static class MapNode {
    public final String path;
    public String leftNeighbor  = null;
    public String rightNeighbor = null;
    public MapNode(String path) { this.path = path; }
}
```

Questo design è deliberatamente orientale: usare stringhe come chiavi invece di riferimenti diretti agli oggetti evita dipendenze circolari e permette di referenziare mappe non ancora caricate in memoria.

Inizializzazione e collegamento

Il collegamento tra le mappe avviene in un blocco statico static { … }, che viene eseguito una volta sola al primo accesso alla classe. Il grafo implementa una topologia lineare orizzontale a quattro nodi:

```
Mappa1 ──▶ Mappa2 ──▶ Mappa3 ──▶ Mappa4
```

La struttura è bidirezionale: ogni nodo dichiara esplicitamente sia il vicino destro che quello sinistro. L'assenza di un vicino (campo null) segnala al gioco che quel bordo è invalicabile.

API pubblica

La classe espone tre metodi statici:

- get(String mapPath) — Restituisce il MapNode associato o null se la mappa non è registrata.
- hasRight(String mapPath) — Verifica se esiste un vicino a destra.
- hasLeft(String mapPath) — Verifica se esiste un vicino a sinistra.

Ruolo nell'architettura

MapGraph è consultata da LevelScreen all'interno del metodo update() ogni frame per verificare se il personaggio ha raggiunto un bordo laterale della mappa. Se il vicino è registrato, viene istanziato un nuovo LevelScreen con la mappa adiacente e viene chiamato BaseGame.setActiveScreen() per effettuare la transizione. La classe è quindi il componente che trasforma una serie di file .tmx indipendenti in un mondo di gioco continuo e navigabile.

5.2 TitleScreen.java — Schermata del titolo

La classe TitleScreen gestisce la schermata animata del titolo, mostrata dopo la schermata di intro. Implementa BaseScreen e si occupa di riprodurre una sequenza di frame pre-renderizzati, consentire lo skip tramite input utente e transitare automaticamente al menu principale.

Struttura e campi principali

```
private static final int   TOTAL_FRAMES   = 240;   // 7s × 30fps
private static final float FPS            = 30f;
private static final float FRAME_DURATION = 1f / FPS;

private float elapsedTime       = 0f;
private int   currentFrameIndex = -1;
private boolean done            = false;
```

L'animazione del titolo è composta da 240 frame (corrispondenti a circa 7 secondi a 30 fotogrammi al secondo), caricati come file PNG individuali dalla directory assets/Titolo/ con la convenzione di nomenclatura frame_XXXX.png.

Gestione delle texture e ottimizzazione della memoria

Un aspetto tecnico rilevante di questa classe è la strategia di caricamento delle texture: invece di pre-caricare tutti i 240 frame in memoria — operazione proibitiva per la VRAM — viene mantenuta una sola texture attiva alla volta:

```
private Texture currentFrame;
```

Il metodo render() calcola l'indice del frame corrente in base al tempo trascorso, e carica il file corrispondente solo quando l'indice cambia. Il frame precedente viene esplicitamente distrutto prima di caricare il successivo:

```
if (frameIndex != currentFrameIndex) {
    String path = String.format("assets/Titolo/frame_%04d.png",
frameIndex);
    if (Gdx.files.internal(path).exists()) {
        Texture next = new Texture(Gdx.files.internal(path));
        if (currentFrame != null)
            currentFrame.dispose();
        currentFrame      = next;
        currentFrameIndex = frameIndex;
    }
}
```

Questa tecnica di lazy loading con dispose immediato è il pattern corretto in libGDX per la riproduzione di sequenze frame-by-frame di lunga durata: ogni oggetto Texture alloca memoria sulla GPU, e non liberarla causerebbe un memory leak progressivo che degraderebbe le prestazioni o crasherebbe l'applicazione.

Gestione dell'input e skip

La classe controlla ogni frame se l'utente ha premuto un tasto o effettuato un tocco:

```
if (Gdx.input.justTouched() ||
Gdx.input.isKeyJustPressed(Input.Keys.ANY_KEY)) {
    goToMenu();
    return;
}
```

Viene usata la constante Input.Keys.ANY_KEY per intercettare qualsiasi pressione da tastiera, rendendo il sistema di skip universale e non legato a tasti specifici.

Transizione e flag di sicurezza

Il metodo goToMenu() utilizza un flag booleano done per prevenire transizioni multiple nel caso in cui l'input venga rilevato in contemporanea con il termine naturale dell'animazione:

```
private void goToMenu() {
    if (done) return;
    done = true;
    BaseGame.setActiveScreen(new MenuScreen());
}
```

Questo pattern è una buona pratica in ambienti event-driven dove più condizioni possono scatenare la stessa azione nella stessa iterazione del game loop.

Utilizzo di libGDX

Nel metodo render(), per disegnare il frame corrente viene utilizzato un SpriteBatch coordinato con il sistema di viewport di BaseScreen. Le coordinate e le dimensioni di disegno sono recuperate dal viewport del mainStage:

- float vw = mainStage.getViewport().getWorldWidth();
- float vh = mainStage.getViewport().getWorldHeight();
- batch.setProjectionMatrix(mainStage.getCamera().combined);
- batch.begin();

- batch.draw(currentFrame, 0, 0, vw, vh);
- batch.end();

In questo modo l'immagine occupa sempre l'intera area visibile indipendentemente dalla risoluzione effettiva dello schermo, comportamento corretto per una schermata di titolo a pieno schermo.

5.3 LevelScreen.java — Transizioni tra mappe e gestione delle texture
5.3.1 Sistema di transizione tra mappe (metodo update(), righe 470–487)
La parte finale del metodo update() in LevelScreen implementa il meccanismo di transizione tra aree di gioco adiacenti. Viene eseguita ogni frame e controlla se il personaggio ha raggiunto i bordi laterali della mappa corrente:

```
// Transizione mappa destra
MapGraph.MapNode node = MapGraph.get(mapPath);
if (node != null && node.rightNeighbor != null) {
    if (pablo.getX() + pablo.getWidth() >=
BaseActor.getWorldBounds().width - 2f) {
        PlayerState.saveFrom(pablo);
        BaseGame.setActiveScreen(new
LevelScreen(node.rightNeighbor, "left"));
    }
}
// Transizione mappa sinistra
if (node != null && node.leftNeighbor != null) {
    if (pablo.getX() <= 2f) {
        PlayerState.saveFrom(pablo);
        BaseGame.setActiveScreen(new
LevelScreen(node.leftNeighbor, "right"));
    }
}
```

La logica opera in tre fasi sequenziali:

1. Consultazione del grafo delle mappe: viene recuperato il nodo corrispondente alla mappa attuale tramite MapGraph.get(mapPath). Se il nodo non esiste o il vicino nella direzione richiesta è null, la transizione non avviene e il bordo è invalicabile.

2. Rilevazione della posizione di uscita: la condizione pablo.getX() + pablo.getWidth() >= BaseActor.getWorldBounds().width - 2f verifica se il bordo destro del personaggio ha superato il bordo destro del mondo

con una tolleranza di 2 pixel. Analogamente per il bordo sinistro con pablo.getX() <= 2f. La tolleranza evita falsi negativi causati da imprecisioni floating point nel posizionamento.

3. Salvataggio dello stato e istanziazione del nuovo livello: prima di effettuare la transizione, PlayerState.saveFrom(pablo) serializza i valori correnti di vita e anima del personaggio in campi statici. Il nuovo LevelScreen riceve come parametro il path della mappa adiacente e la stringa "left" o "right" che indica da quale lato il personaggio deve apparire nella nuova mappa. All'interno del costruttore del nuovo livello, PlayerState.applyTo(pablo) ripristina i valori salvati sul nuovo oggetto Pablo istanziato.

Questo meccanismo garantisce continuità dell'esperienza di gioco: il personaggio mantiene vita e anima attraverso le transizioni, e appare sul bordo corretto della nuova mappa.

5.3.2 Il metodo loadTextureSafe

```
private Texture loadTextureSafe(String path, Color fallback) {
    try {
        return new Texture(Gdx.files.internal(path));
    } catch (Exception e) {
        Gdx.app.log("LevelScreen", "Asset non trovato: " + path);
        Pixmap pix = new Pixmap(1, 1, Pixmap.Format.RGBA8888);
        pix.setColor(fallback);
        pix.fill();
        Texture t = new Texture(pix);
        pix.dispose();
        return t;
    }
}
```

Questo metodo di utilità incapsula il caricamento di una texture con gestione degli errori integrata. Se il file specificato dal path non esiste o non è leggibile, invece di propagare un'eccezione non gestita che causerebbe il crash dell'applicazione, il metodo genera programmaticamente una texture di dimensione 1×1 pixel con il colore di fallback specificato.
Il processo di creazione della texture di fallback avviene tramite Pixmap, una struttura dati in memoria CPU che rappresenta un'immagine. Dopo aver creato la texture dalla Pixmap, quest'ultima viene immediatamente

rilasciata con pix.dispose() per liberare la memoria CPU, poiché la texture stessa risiede ormai sulla GPU.
L'uso di questo pattern è particolarmente rilevante in fase di sviluppo e testing: permette al gioco di avviarsi e funzionare correttamente anche in presenza di asset grafici mancanti, mostrando un colore di placeholder chiaramente visibile invece di un crash opaco.

5.3.3 Il metodo createEmptyVesselTexture

```
private Texture createEmptyVesselTexture(int size) {
    Pixmap pix = new Pixmap(size, size, Pixmap.Format.RGBA8888);
    pix.setColor(0f, 0f, 0f, 0f);
    pix.fill();
    int cx     = size / 2;
    int cy     = size / 2;
    int radius = Math.max(1, (size / 2) - 2);
    pix.setColor(0.12f, 0.12f, 0.12f, 1f);
    pix.fillCircle(cx, cy, radius - 2);
    pix.setColor(0.35f, 0.35f, 0.35f, 1f);
    pix.drawCircle(cx, cy, radius - 1);
    Texture t = new Texture(pix);
    pix.dispose();
    return t;
}
```

Questo metodo genera programmaticamente la texture del ricettacolo dell'anima (soul vessel) nello stato vuoto, senza necessità di un file immagine esterno. La creazione avviene interamente in memoria tramite operazioni su Pixmap:

- Prima l'intera area è riempita con trasparenza totale (alpha = 0).
- Poi viene disegnato un cerchio pieno con un grigio molto scuro (0.12, 0.12, 0.12) come base del ricettacolo.
- Infine viene sovrapposto un cerchio wireframe leggermente più chiaro (0.35, 0.35, 0.35) come bordo.

La generazione procedurale di questa texture evita di dover includere nel progetto un file asset per un elemento grafico semplice, e ne garantisce la scalabilità: il parametro size permette di generare la texture alla dimensione corretta per qualsiasi risoluzione di HUD.

5.4 FalseKnight.java — Intelligenza artificiale e animazioni del boss

FalseKnight è il nemico boss del gioco, il più complesso dell'intera codebase per numero di stati, animazioni e meccaniche. La classe non estende Enemy ma direttamente BaseActor, poiché il boss richiede un controllo più fine sulla fisica e sul comportamento rispetto ai nemici comuni.

5.4.1 Caricamento delle animazioni (loadAnimations)

Il metodo loadAnimations() viene chiamato dal costruttore e definisce tutte le animazioni del boss tramite il metodo loadAnimationFromFiles ereditato da BaseActor. Sono presenti tredici animazioni distinte che coprono ogni fase del comportamento:

Animazione	Descrizione	PlayMode
animIdle	Respirazione in loop	LOOP
animTurn	Frame singolo di inversione	NORMAL
animRunAnticipate	Anticipazione della corsa	NORMAL
animRun	Ciclo di corsa	LOOP
animAttackAnticipate	Preparazione attacco con frame duplicati per simulare una pausa	NORMAL
animAttack	Tre frame di attacco rapido	NORMAL
animAttackRecover	Recupero post-attacco	NORMAL
animJumpAnticipate	Preparazione al salto	NORMAL
animJumpUp	Fase ascendente del salto	NORMAL
animJumpAttackFall	Caduta con attacco dall'alto	NORMAL
animJumpLand	Atterraggio	NORMAL
animStunRoll	Stagger dopo danno	NORMAL
animStunEnd	Fine stagger e morte	NORMAL

Un elemento tecnico significativo nel caricamento di animAttackAnticipate è la duplicazione del frame finale per simulare una pausa visiva:

```
animAttackAnticipate = loadAnimationFromFiles(new String[]{
    PATH+"attack(anticipate)5.png",
    PATH+"attack(anticipate)5.png",
    PATH+"attack(anticipate)5.png"
}, 0.050f, false);
```

Triplicando il fotogramma 5 si ottiene un effetto di congelamento di 150ms senza dover implementare logica di pausa separata, sfruttando direttamente il sistema di animazione di libGDX.

5.4.2 Macchina a stati e metodi tick

Il comportamento del boss è interamente governato da un enum FalseKnightState con dodici stati. Il dispatching avviene nel metodo act() tramite uno switch:

```
switch (state) {
    case IDLE:               tickIdle();              break;
    case PATROL:             tickPatrol();            break;
    case TURN:               tickTurn();              break;
    case ANTICIPATE_ATTACK:  tickAnticipateAttack();  break;
    case ATTACK:             tickAttack();            break;
    case RECOVER_ATTACK:     tickRecoverAttack();     break;
    case JUMP_ANTICIPATE:    tickJumpAnticipate();    break;
    case JUMP_UP:            tickJumpUp();            break;
    case JUMP_ATTACK_FALL:   tickJumpAttackFall();    break;
    case JUMP_LAND:          tickJumpLand();          break;
    case STAGGER:            tickStagger();           break;
    case DEAD:               tickDead();              break;
}
```

Di seguito l'analisi dei principali metodi tick:
tickIdle(): Il boss rimane fermo con l'animazione di respirazione. Dopo un tempo definito da IDLE_DURATION (1.5 secondi), transita automaticamente allo stato PATROL. Questo stato funge da reset comportamentale tra gli attacchi.
tickPatrol(): Implementa la logica di inseguimento. Il boss si orienta verso Pablo tramite updateFacingTowardPablo() e si muove verso di lui a velocità PATROL_SPEED se è nell'intervallo di aggro e non è vicino a un bordo. La decisione di quale attacco eseguire è basata sulla distanza:

```
if (inAggro && attackCooldown <= 0f && stateTimer > 0.5f) {
    if (dist <= JUMP_ATTACK_DIST) {
        enterState(FalseKnightState.ANTICIPATE_ATTACK);
    } else {
        enterState(FalseKnightState.JUMP_ANTICIPATE);
    }
    attackCooldown = ATTACK_COOLDOWN;
}
```

Se la distanza è inferiore a 250 pixel, viene eseguito l'attacco a terra; altrimenti il boss tenta un salto per raggiungere il giocatore da lontano.
tickAttack(): Gestisce l'attacco a terra. La hitbox della mazza viene attivata dopo 60ms dall'inizio dello stato. In contemporanea viene triggerato un hitstop e uno screen shake tramite callback:

```
if (stateTimer >= 0.060f && !hitStopFired) {
    triggerHitStop(0.075f);
    triggerScreenShake();
    hitStopFired = true;
}
```

Il flag hitStopFired previene la ripetizione del callback in ogni frame successivo al primo.
tickJumpAttackFall(): Durante la caduta dell'attacco in salto, viene applicata una gravità supplementare FALL_ACCEL per enfatizzare l'impatto visivo della discesa. La hitbox rimane attiva durante tutta la fase di caduta, rendendo l'attacco pericoloso per un arco di tempo prolungato.
tickJumpLand(): All'atterraggio, la mazza viene disattivata, lo stato del personaggio si azzera e viene ripetuto hitstop e screen shake:

```
if (!hitStopFired) {
    triggerHitStop(0.075f);
    triggerScreenShake();
    hitStopFired = true;
}
```

tickDead(): Il boss subisce un fade-out progressivo prima di essere rimosso dallo Stage:

```
setOpacity(Math.max(0f, getColor().a - 0.005f));
if (getColor().a <= 0f) remove();
```

tickStagger(): Implementa la reazione al danno subito. Il boss arretra con velocità STAGGER_SPEED nella direzione opposta al proprio orientamento, poi reproduce l'animazione di fine stagger prima di tornare in IDLE.

5.4.3 Sincronizzazione animazione-gameplay

Un aspetto critico dell'implementazione è la gestione del metodo enterState():

```
private void enterState(FalseKnightState next) {
    state         = next;
    stateTimer    = 0f;
    hitStopFired  = false;
    elapsedTime   = 0f;
    maceActive    = false;
}
```

Il reset di elapsedTime (campo di BaseActor che pilota il metodo getKeyFrame() dell'animazione) garantisce che ogni cambio di stato riparta sempre dal primo frame dell'animazione associata. Senza questo reset, un'animazione potrebbe partire da metà sequenza se lo stato precedente era durato a lungo, producendo artefatti visivi.
La sincronizzazione del facing sprite avviene tramite syncFacingToHorizontalMovement():

```
private void syncFacingToHorizontalMovement() {
    if (velocityVec.x > 0) { setScaleX(1f); facingDir = 1; }
    else if (velocityVec.x < 0) { setScaleX(-1f); facingDir = -1;
}
    else { faceDirection(facingDir); }
}
```

Applicare setScaleX(-1f) è il metodo standard in libGDX per eseguire un flip orizzontale dello sprite senza dover caricare versioni separate delle texture per ogni direzione.

5.5 Pablo.java — Animazione del salto (updateJumpAnimation)

Il metodo updateJumpAnimation() è responsabile della scelta del frame di animazione corretto durante le fasi di salto e caduta del personaggio. La sua implementazione divide la parabola di salto in segmenti temporali proporzionali, mappando ciascuno a un frame specifico.

```
private void updateJumpAnimation() {
    float p = jumpTimer / totalJumpTime;
    if (p < 0.10f)
        setAnimation(jumpUp);
    else if (p < 0.50f) {
        float r = (p - 0.10f) / 0.40f;
        if (r < 0.40f) setAnimation(frameRise1);
        else if (r < 0.80f) setAnimation(frameRise2);
        else setAnimation(frameAltezzaMax);
    } else if (p < 0.90f) {
        float d = (p - 0.50f) / 0.40f;
        if (d < 0.20f) setAnimation(frameAltezzaMax);
        else if (d < 0.60f) setAnimation(frameDrop1);
        else setAnimation(frameDrop2);
    } else {
        setAnimation(frameDrop2);
    }
}
```

Il parametro p rappresenta la progressione normalizzata del salto: è il rapporto tra il tempo trascorso dall'inizio del salto (jumpTimer) e la durata teorica totale (totalJumpTime), calcolata nel costruttore come (jumpSpeed / gravity) * 2.0f. Il valore di p è quindi sempre compreso nell'intervallo [0.0, 1.0] per un salto completo. La suddivisione in fasce è la seguente:

Intervallo di p	Fase del salto	Animazione
p < 0.10	Decollo (primo 10%)	jumpUp (3 frame multipli)
0.10 ≤ p < 0.50	Fase ascendente	frameRise1, frameRise2, frameAltezzaMax
0.50 ≤ p < 0.90	Fase discendente	frameAltezzaMax, frameDrop1, frameDrop2
p ≥ 0.90	Caduta finale (ultimo 10%)	frameDrop2

All'interno delle fasce intermedie, la variabile locale r (per la salita) e d (per la discesa) normalizzano ulteriormente la progressione all'interno del sotto-intervallo, permettendo di suddividere equamente i frame disponibili. Ad esempio, nella fase ascendente, frameRise1 copre il 40% della durata ascendente, frameRise2 il successivo 40%, e frameAltezzaMax il 20% finale prima dell'apice.
Questa soluzione è preferibile all'uso di un'animazione libGDX standard con PlayMode.NORMAL perché offre controllo granulare su ogni singola fase: il designer può decidere di dedicare più o meno tempo a ciascun frame semplicemente modificando le soglie numeriche, senza dover ricreare i file di animazione. Inoltre, l'animazione rimane sincronizzata con la fisica reale del salto piuttosto che con un timer fisso, adattandosi correttamente a eventuale modifiche di jumpSpeed o gravity.

---

6. Funzionamento del gioco

6.1 Game loop e schermate

Il game loop è gestito da libGDX attraverso l'interfaccia Screen. Ogni schermata estende BaseScreen, che nel metodo render() esegue in sequenza: applicazione dei viewport, avanzamento degli stage (act()), aggiornamento della logica di gioco (update()), pulizia del buffer colore e disegno degli stage (draw()).
La navigazione tra schermate avviene tramite BaseGame.setActiveScreen(), che delega al metodo setScreen() della classe Game di libGDX. Quando una schermata diventa inattiva, hide() rimuove i suoi input processor dall'InputMultiplexer; quando diventa attiva, show() li reintegra.

6.2 Fisica del personaggio

Il personaggio Pablo implementa fisiche personalizzate senza l'uso di motori fisici esterni come Box2D. Le forze vengono accumulate nel vettore accelerationVec e applicate a velocityVec ogni frame, il quale viene poi integrato nella posizione con moveBy(). La gravità viene applicata sottraendo un valore costante dalla componente y della velocità, clamped a maxVerticalSpeed per evitare velocità di caduta illimitate.
Le collisioni con i solidi vengono gestite in LevelScreen.update() tramite il metodo preventOverlap() di BaseActor, che calcola il Minimum Translation Vector (MTV) tra i poligoni di collisione usando Intersector.overlapConvexPolygons(). La componente dominante del MTV determina se il contatto è orizzontale (muro) o verticale (pavimento/soffitto), con conseguente azzeramento della velocità nel relativo asse.

6.3 Sistema di nemici
Tutti i nemici condividono la classe base Enemy, che fornisce: gravità, sensori di pavimento e bordo, metodi flip() e faceDirection() per la gestione del facing, metodo takeDamage() base e il sistema di attivazione basato sulla prossimità della camera (isActiveNearCamera()). Quest'ultimo è fondamentale per le prestazioni: i nemici lontani dalla camera vengono congelati, evitando calcoli fisici e AI superflui per oggetti non visibili.

---

7. Gestione grafica e audio
7.1 Sistema di animazioni
Tutte le animazioni sono gestite tramite la classe Animation<TextureRegion> di libGDX, popolata con TextureRegion caricati da file PNG individuali. Il metodo getKeyFrame(elapsedTime) seleziona automaticamente il frame corretto in base al tempo trascorso e alla modalità di riproduzione (LOOP o NORMAL). Il campo elapsedTime di BaseActor viene incrementato nel metodo act() e resettato a 0f in enterState() per sincronizzare ogni animazione con il suo stato di ingresso.

7.2 SoundManager
La classe SoundManager è un Singleton che gestisce tutta l'audio del gioco. Le tracce musicali (Music) sono ottimizzate per brani lunghi con streaming da disco; gli effetti sonori (Sound) sono caricati interamente in memoria per minimizzare la latenza di riproduzione. Il sistema implementa un meccanismo di cooldown per SFX: ogni effetto sonoro ha un tempo minimo tra due riproduzioni consecutive per evitare accumuli indesiderati (ad esempio, il suono di salto non può riprodursi più di una volta ogni 100ms). Il metodo update(float dt) deve essere chiamato ogni frame per decrementare i timer di cooldown.
Il volume di musica ed effetti è controllato in tempo reale tramite slider nella schermata opzioni, con propagazione immediata ai canali audio attivi attraverso i setter di GameSettings che a loro volta invocano i setter di SoundManager.

7.3 Rendering della UI
L'interfaccia utente in LevelScreen utilizza un Stage separato (uiStage) con ScreenViewport, che garantisce che gli elementi di HUD (maschere di vita, ricettacolo dell'anima) siano sempre renderizzati a dimensioni in pixel reali indipendenti dalla risoluzione del mondo di gioco. Il layout è costruito tramite la classe Table di scene2d, che gestisce automaticamente l'allineamento e la spaziatura degli elementi.

8. Problemi affrontati e soluzioni adottate

8.1 Consumo di memoria nelle sequenze frame-by-frame

La riproduzione di sequenze animate composte da centinaia di frame PNG (intro: 424 frame, titolo: 240 frame) avrebbe saturato la VRAM se tutti i frame fossero stati caricati simultaneamente. La soluzione adottata è il caricamento singolo lazy: una sola Texture viene mantenuta in memoria alla volta, il frame precedente viene esplicitamente distrutto con dispose() prima di caricare il successivo, e il controllo Gdx.files.internal(path).exists() previene crash in caso di frame mancanti nel filesystem.

8.2 Gestione degli asset mancanti

L'adozione sistematica di loadTextureSafe() in LevelScreen garantisce che la mancanza di un file PNG non causi crash durante la fase di inizializzazione. La texture di fallback generata programmaticamente tramite Pixmap permette al gioco di avviarsi e funzionare anche con asset parziali, semplificando lo sviluppo iterativo.

8.3 Sincronizzazione del facing del boss

Un bug iniziale nel boss causava discrepanze tra la direzione di movimento e la direzione dello sprite, poiché il campo facingDir non veniva aggiornato insieme a setScaleX(). La soluzione adottata è stata unificare le due operazioni nel metodo syncFacingToHorizontalMovement():

```
private void syncFacingToHorizontalMovement() {
    if (velocityVec.x > 0) { setScaleX(1f); facingDir = 1; }
    else if (velocityVec.x < 0) { setScaleX(-1f); facingDir = -1;
}
    else { faceDirection(facingDir); }
}
```

8.4 Iniezione della gravità nel boss durante la caduta in attacco

Durante lo stato JUMP_ATTACK_FALL, la gravità standard applicata in applyGravity() veniva sovrastata da una gravità customizzata più aggressiva (FALL_ACCEL). Applicare entrambe in contemporanea produceva velocità di caduta irrealistiche. La soluzione è stata aggiungere un guard clause in applyGravity() per saltare il calcolo se lo stato corrente lo gestisce autonomamente:

```
private void applyGravity(float dt) {
    if (state == FalseKnightState.JUMP_ATTACK_FALL) return;
    // ...
}
```

8.5 Attivazione prematura dei nemici

I nemici venivano istanziati e attivati fisicamente al caricamento dell'intera mappa, causando comportamenti errati (caduta fuori dalla mappa, accumulo in angoli non visibili) prima che il giocatore raggiungesse la loro area. Il sistema isActiveNearCamera() in Enemy risolve questo problema congelando completamente il nemico (azzerando velocityVec) quando la distanza dalla camera supera il margine di attivazione di 320 pixel.

---

9. Considerazioni finali

Il progetto Pablo dimostra una progettazione software matura e consapevole, costruita attorno alle peculiarità del framework libGDX. L'adozione di pattern consolidati come il Singleton, la macchina a stati e il lazy loading delle risorse riflette una comprensione profonda delle sfide specifiche dello sviluppo di videogiochi: gestione della memoria grafica, mantenimento di prestazioni stabili, sincronizzazione tra logica e rendering.

La separazione tra BaseActor e le classi concrete, tra BaseScreen e le schermate specifiche, e tra MapGraph e la logica di navigazione permette un alto grado di estendibilità: aggiungere un nuovo tipo di nemico richiede solo l'implementazione di una sottoclasse di Enemy con i propri stati e animazioni; aggiungere una nuova mappa al grafo dei livelli richiede sole quattro righe di codice in MapGraph.

Possibili evoluzioni future

- Sistema di salvataggio persistente: integrare le Preferences di libGDX o serializzazione JSON per salvare progressi, checkpoint e impostazioni tra sessioni.
- Aggiunta di nuovi livelli: la struttura di MapGraph supporta nodi ulteriori con topologie non lineari (ramificazioni verticali, loop), che potrebbero arricchire significativamente l'esplorazione.
- Ottimizzazione asset: adottare un AssetManager libGDX con caricamento asincrono per eliminare i micro-freeze durante il caricamento delle texture nelle transizioni tra mappe.
- Sistema di dialogo: il dialogo del boss potrebbe essere esteso in un sistema di dialogo generico, reutilizzabile per NPC e narrativa ambientale.
- Porting multipiattaforma: libGDX supporta nativamente Android, iOS e HTML5; la codebase, già strutturata senza dipendenze da API desktop-specifiche nel game logic, è ben posizionata per un porting su piattaforme mobile.